

Algoritmy a grafy 1 (BI-AG1), Přednáška č. 2

Souvislost, složitost, stromy

Dušan Knop, Michal Opler, Tomáš Valla

Fakulta informačních technologií
České vysoké učení technické v Praze
<https://courses.fit.cvut.cz/BI-AG1/>



ZS 2025/2026, 29. září 2025

(Verze dokumentu: 26. 1. 2026 08:52)

Nejvýš jedna modrá hrana

Vstup

Neorientovaný graf $G = (V, E)$, vrchol $s \in V$. Rozdělení hran E na množiny červených R a modrých B hran (tj. $E = R \cup B$ a $R \cap B = \emptyset$).

Cestu z s do v v grafu G nazveme **přípustnou**, pokud používá nanejvýš jednu modrou hranu.

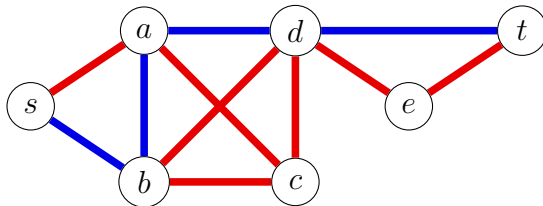
Výstup

- Pole vzdáleností D takové, že

$$D[v] = \begin{cases} k & k \text{ je délka nejkratší přípustné } s\text{-}v\text{-cesty v } G \\ \text{undef.} & \text{neexistuje žádná přípustná } s\text{-}v\text{-cesta v } G \end{cases}$$

- Pole předchůdců P pro (případné) přípustné nejkratší cesty v G .

Nejvýš jedna modrá hrana



Přípustné s - t -cesty:

- s, a, d, e, t
- s, b, d, e, t
- s, a, c, d, t
- s, a, c, d, e, t

Jak vypadají řešení?

Kombinace tří cest – P, Q a R v grafu G (mohou být prázdné) takových, že

- P a R jsou tvořeny pouze červenými hranami
- cesta Q obsahuje nanejvýš jednu hranu a ta je v takovém případě modrá

Nejvýš jedna modrá hrana

Jak vypadají řešení?

Kombinace tří cest – P, Q a R v grafu G (mohou být prázdné) takových, že

- P a R jsou tvořeny pouze červenými hranami
- cesta Q obsahuje nanejvýš jednu hranu a ta je v takovém případě modrá

Modifikace grafu

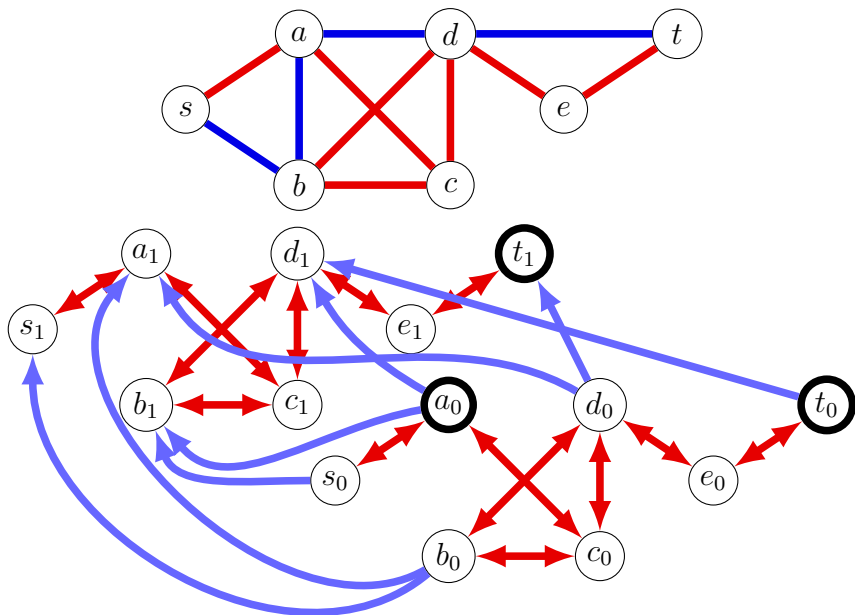
Zavedeme nový **orientovaný** graf H .

Pro každý vrchol $v \in V(G)$ uděláme jeho čistou a červenou kopii (označme je $v_0 \in V(H)$ a $v_1 \in V(H)$).

Hrany grafu H budou následující:

- 1 (u_0, v_0) a (u_1, v_1) pokud $\{u, v\} \in E_R$ a
- 2 (u_0, v_1) pokud $\{u, v\} \in E_B$.

Nejvýš jedna modrá hrana



Nejvýš jedna modrá hrana – Korektnost

Věta

Přípustné s - v -cesty v grafu G odpovídají s - v -cestám v grafu H a naopak.

Algoritmus BFS, hledající nejkratší s_0 - t cestu, spuštěný na grafu H buď nikdy neopustí červené „nulové“ hrany a pak skončí ve v_0 nebo někdy použije modrou hranu a poté už smí používat jen červené „jedničkové“ hrany a skončí ve v_1 .

Důkaz.

- Přípustnou s - v -cestu v G lze rozložit na podcesty P, Q a R a ty lze pak najít v grafu H .
- Libovolnou orientovanou cestu v grafu H lze rozložit na tři cesty, které lze spojit do jednoho sledu v grafu G .
- Kdyby výsledný sled nebyl cestou, lze původní cesty zkrátit, což je ve sporu s korektností algoritmu BFS. □

Příklad, k čemu lze nejkratší cesty v grafu použít

Netriviální příklad (který nevypadá jako grafová úloha):

Úloha

- Je dáno přirozené číslo $K > 1$.
- Chceme najít přirozené číslo $N \geq 1$ s **nejmenším počtem cifer** (tj. co nejkratší číslo), které
 - ▶ má v **desítkovém** zápise pouze číslice 0 a 1 a
 - ▶ je **dělitelné** K ,případně oznámit, že takové číslo neexistuje.

Protože $N \geq 1$, je zřejmé, že N musí začínat číslicí 1.

Nejkratší číslo v dekadické soustavě s binárními číslicemi dělitelné

- číslem $K = 3$ je $N = 111$
- číslem $K = 4$ je $N = 100$
- číslem $K = 5$ je $N = 10$
- číslem $K = 6$ je $N = 1110$

Řešení příkladu – obecné

- Sestrojíme orientovaný graf $G = (V, E)$ nad **zbytky po dělení číslem K** , tedy
 - ▶ $V = \{0, \dots, K - 1\}$,
 - ▶ z vrcholu $Z \in V$ vedou právě dvě hrany pro číslici 0 a pro číslici 1 po řadě do vrcholů $(10Z + 0) \bmod K$ a $(10Z + 1) \bmod K$.

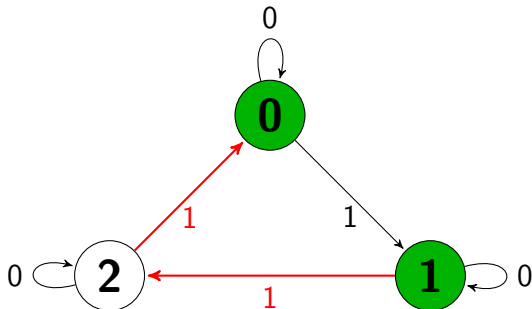
Fakt

Graf vyjadřuje skutečnost, že když máme aktuální číslo X takové, že $X \bmod K = Z$, pak pro číslo XC vzniklé připsáním číslice C na konec za číslo X platí, že $XC \bmod K = ZC \bmod K$ a není proto nutné uchovávat celé číslo X a stačí vzít zbytek Z .

- Číslice nejmenšího čísla N dělitelného K leží tedy na nejkratší cestě v tomto grafu začínající ve vrcholu 1 (číslo N začíná číslicí 1) a končící ve vrcholu 0.
- Tu lze najít standardním algoritmem **BFS** pro orientované grafy. Lze snadno zobecnit na širší množinu číslic než pouze 0 a 1.

Řešení příkladu – $K = 3$

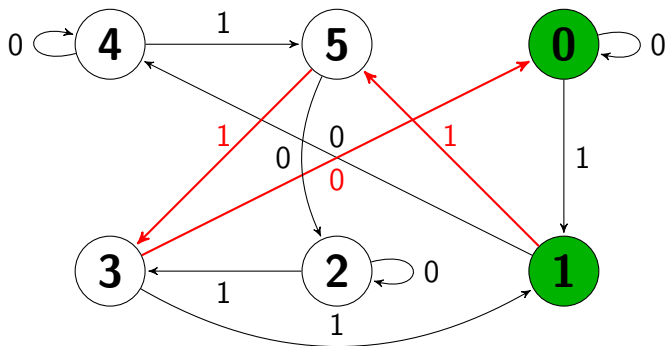
Ukažme si řešení pro $K = 3$. Pak $V = \{0, 1, 2\}$ je množina zbytkových tříd a orientovaný graf je díky skutečnosti, že $10 \bmod 3 = 1$, velmi speciální.



Nejkratší cesta z vrcholu 1 do vrcholu 0 má značky 11, čili nejkratší číslo v dekadické soustavě s binárními číslicemi dělitelné 3 je $N = 111$.

Řešení příkladu – $K = 6$

Ukažme si řešení pro $K = 6$. Pak $V = \{0, 1, 2, 3, 4, 5\}$ je množina zbytkových tříd a orientovaný graf je:



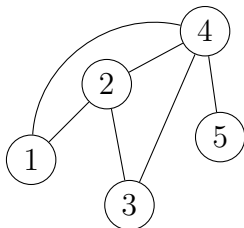
Nejkratší cesta z vrcholu 1 do vrcholu 0 má značky 110, čili nejkratší číslo v dekadické soustavě s binárními číslicemi dělitelné 6 je $N = 1110$.

Souvislost grafu

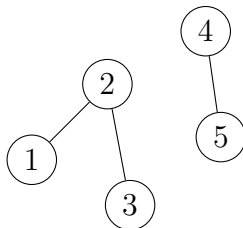
Definice

Graf G je **souvislý**, jestliže v něm pro každé jeho dva vrcholy u, v existuje u - v -cesta.

Jinak je G **nesouvislý**.



(a) Souvislý graf



(b) Nesouvislý graf

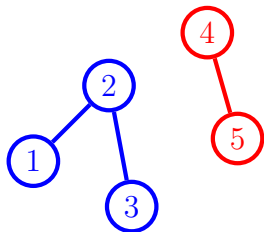
Souvislé komponenty grafu I

Definice

Indukovaný podgraf H grafu G je **souvislou komponentou**, pokud

- je souvislý a
- neexistuje žádný souvislý podgraf F , $F \neq H$, grafu G takový, že $H \subseteq F$.

(Souvislá komponenta je tedy **v inkluzi maximální souvislý podgraf** grafu G .)



Souvislé komponenty grafu II

Z definic souvislého grafu a souvislé komponenty okamžitě plyne:

Pozorování

Graf je souvislý právě tehdy, když obsahuje jedinou souvislou komponentu.

Snadno lze ověřit, že:

Tvrzení

Binární relace $\rightsquigarrow$ nad vrcholy grafu G definovaná předpisem

$$u \rightsquigarrow v \iff \exists \text{ } u\text{-}v\text{-cesta v } G$$

je ekvivalence, jejíž třídy ekvivalence indukují souvislé komponenty grafu G .

Algoritmy pro vyšetřování souvislosti grafu

Úloha vyšetřování souvislosti grafu G má různé podoby, např.:

- (a) Zjistit, zda je G souvislý.
- (b) Pokud není, identifikovat všechny souvislé komponenty.
- (c) Je-li zadán vrchol s , zkonstruovat souvislou komponentu, do které s patří.

Vstup: Graf G a jeden jeho vrchol s .

Výstup: Graf s označenou souvislou komponentou, ve které leží vrchol s .

Vstup: Graf G a jeden jeho vrchol s .

Výstup: Graf s označenými souvislými komponentami.

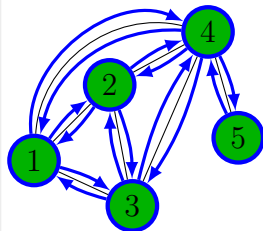
Algoritmus **BFS_graf** (graf G):

- (1) Pro každý vrchol $u \in V(G)$:
- (2) stav[u] := nenalezený
- (3) D[u] := P[u] := undef
- (4) Pro každý vrchol $s \in V(G)$:
- (5) Pokud stav[s] = nenalezený:
- (6) BFS(G , s)
- (7) Vrať (D, P)

BFS (graf G , vrchol s):

- (8) Q := fronta obsahující s
- (9) stav[s] := otevřený, D[s] := 0, P[s] := $\perp$
- (10) Dokud je fronta Q neprázdná:
- (11) Odeber z Q první vrchol $\rightarrow v$
- (12) Pro všechny sousedy w vrcholu v :
- (13) Pokud stav[w] = nenalezený:
- (14) stav[w] := otevřený
- (15) D[w] := D[v] + 1, P[w] := v
- (16) Přidej w na konec Q
- (17) stav[v] := uzavřený

1	→	4	3	2	
2	→	3	1	4	
3	→	4	2	1	
4	→	5	3	1	2
5	→	4			



Alternativa: DFS

Algoritmus **DFS_graf** (graf G , vrchol s):

(1) Pro každý vrchol $u \in V(G)$:

(2) stav[u] := nenalezený

(3) DFS(s)

DFS (vrchol v):

(4) Pokud stav[v] $\neq$ nenalezený:

(5) return

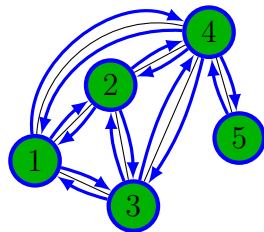
(6) stav[v] := otevřený //žlutý

(7) Pro každého souseda u vrcholu v :

(8) DFS(u)

(9) stav[v] := uzavřený //zelený

1	→	4	3	2	
2	→	3	1	4	
3	→	4	2	1	
4	→	5	3	1	2
5	→	4			



Výpočetní model RAM

- RAM je zkratka *Random Access Machine*.
- **Paměť** RAMu tvoří pole celočíselných paměťových buněk adresovatelných celými čísly.
- **Program** je konečná posloupnost sekvenčně prováděných instrukcí dvou typů: aritmeticko-logických a řídicích.
- **Aritmeticko-logické instrukce** mají obvykle dva vstupní argumenty a jeden výstupní argument. Argument může být
 - ▶ přímá konstanta (s výjimkou výstupního argumentu),
 - ▶ přímo adresovaná paměťová buňka (adresa je přímo zadána),
 - ▶ nepřímá adresovaná paměťová buňka (její adresa je uložena v přímo adresované buňce).
- **Řídicí instrukce** jsou skoky (na konkrétní instrukci programu), podmíněné skoky (například když se dva argumenty instrukce rovnají) a instrukci zastavení programu.

Výpočetní model RAM

- Na začátku výpočtu obsahuje paměť v určených buňkách vstup a obsah ostatních buněk je nedefinován.
- Potom je program sekvenčně prováděn, instrukce za instrukcí.
- Procesor v každém kroku provede právě jednu instrukci.
- Po zastavení programu je obsah paměti v určených buňkách interpretován jako výstup programu.

Složitost programu

Definice

- **Velikost vstupu** je počet paměťových buněk, které vstup zabírá.
- **Časová složitost programu** (v nejhorším případě) pro vstup velikosti N je maximum z počtu vykonaných instrukcí přes všechny možné (~~povolené~~¹) vstupy velikosti nejvýše N .
- **Paměťová složitost programu** (v nejhorším případě) pro vstup velikosti N je maximum z počtu použitých paměťových buněk přes všechny možné (~~povolené~~¹) vstupy velikosti nejvýše N .

¹Pro první tři přednášky si vystačíme se všemi vstupy a později se vrátíme k tomu, co znamenají povolené vstupy.

Reprezentace grafu – Matice sousednosti

Jak reprezentovat (neorientované) grafy v paměti počítače?

- a) **Matice sousednosti**
- b) Seznam sousedů

Definice

Nechť $G = (V, E)$ je graf s $V = \{v_1, v_2, \dots, v_n\}$.

Matice sousednosti grafu G je čtvercová matice $A_G = (a_{ij})_{i,j=1}^n$ definovaná předpisem

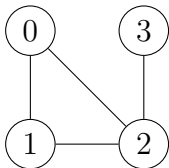
$$a_{i,j} = \begin{cases} 1 & \text{když } \{v_i, v_j\} \in E, \\ 0 & \text{jinak.} \end{cases}$$

Je evidentní, že A_G je symetrická matice.

Paměťová složitost: $O(n^2)$.

Reprezentace grafu – Matice sousednosti

$$G = (\{0, 1, 2, 3\}, \{\{0, 1\}, \{0, 2\}, \{1, 2\}, \{2, 3\}\})$$



(a) Graf G

	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

(b) Matice sousednosti A_G

Reprezentace grafu – Seznam sousedů

Jak reprezentovat grafy v paměti počítače?

a) Matice sousednosti

b) **Seznam sousedů**

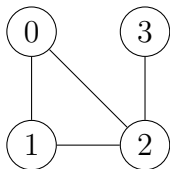
Pro každý vrchol v grafu $G = (V, E)$ uchováváme seznam jeho sousedů (např. spojový seznam).

Paměťová složitost: $|V|$ začátků seznamů a dohromady $2|E|$ prvků v seznamech (každá hrana znamená dva výskyty).

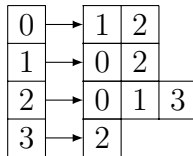
Celkem tedy paměťová složitost: $O(n + m)$.

Reprezentace grafu – Seznam sousedů

$$G = (\{0, 1, 2, 3\}, \{\{0, 1\}, \{0, 2\}, \{2, 1\}, \{3, 2\}\})$$



(a) Graf G

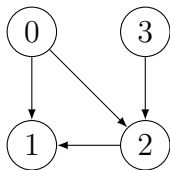


(b) Seznam sousedů

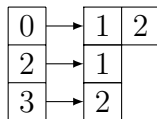
Reprezentace orientovaného grafu

Orientované grafy reprezentujeme ve stejných strukturách jako neorientované grafy, tedy maticí sousednosti nebo seznamem sousedů, pouze místo seznamu sousedů říkáme **seznam následníků**:

$$G = (\{0, 1, 2, 3\}, \{(0, 1), (0, 2), (2, 1), (3, 2)\})$$



	0	1	2	3
0	0	1	1	0
1	0	0	0	0
2	0	1	0	0
3	0	0	1	0



(a) Orientovaný graf G (b) Matice sousednosti A_G (c) Seznam následníků

Analýza časové složitosti BFS

Věta (o časové složitosti algoritmu BFS)

Algoritmus **BFS**(G, s) má při reprezentaci grafu G seznamem sousedů časovou složitost $O(|V| + |E|)$.

Důkaz.

- Každý vrchol je přidán do fronty Q nejvýše jednou, neboť je předtím otevřen a nemůže už tedy vícekrát splnit podmínku na řádku (13), že je nenalezený.
- V každé iteraci cyklu (10)–(17) je jeden vrchol odebrán z fronty, tento cyklus má tedy nejvýše $|V|$ iterací.
- Celkový počet iterací cyklu (12)–(16) je nejvýše dvakrát tolik, kolik je v grafu hran (jednou „pohled“ hranou z jedné strany, podruhé z opačné).
- Celková časová složitost je tedy $O(|V| + |E|)$. □

Analýza časové složitosti BFS (rozklad)

Věta (o časové složitosti algoritmu `BFS_graf`)

Algoritmus `BFS_graf`(G) má při reprezentaci grafu G seznamem sousedů časovou složitost $O(|V| + |E|)$.

Důkaz.

- V globálním seznamu stavu vrcholů se každý vrchol vyjme nanejvýš jednou a poté se pustí `BFS` z něho.
- Každý vrchol je přidán do fronty Q nejvýše jednou.
- V každé iteraci cyklu (10)–(17) je jeden vrchol odebrán z fronty, tento cyklus má tedy nejvýše $|V|$ iterací.
- Celkový počet iterací cyklu (12)–(16) je nejvýše dvakrát tolik, kolik je v grafu hran (neboť pro každý vrchol procházíme jen hrany a vrcholy v jeho komponentě souvislosti).
- Celková časová složitost je tedy $O(|V| + |E|)$. □

Slabá souvislost orientovaných grafů

- Pojmy souvislost a souvislá komponenta neorientovaných grafů **nejdou** jednoduše použít pro orientované grafy.

Definice

Symetrizace orientovaného grafu $G = (V, E)$ je neorientovaný graf $\text{sym}(G) = (V, E')$, kde $\{u, v\} \in E'$ právě tehdy, když $(u, v) \in E$ nebo $(v, u) \in E$.

Symetrizace tedy odstraní z grafu informace o orientaci hran.

Definice

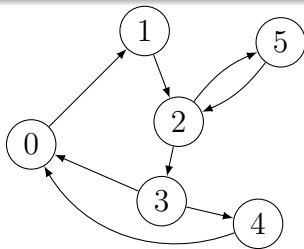
Orientovaný graf $G = (V, E)$ nazveme **slabě souvislý**, pokud je souvislá jeho symetrizace $\text{sym}(G)$.

Testovat slabou souvislost orientovaného grafu tedy znamená testovat, např. pomocí algoritmu **BFS**, souvislost jeho symetrizace, což je neorientovaný graf.

Silná souvislost orientovaných grafů

Definice

Orientovaný graf $G = (V, E)$ nazveme **silně souvislý**, pokud pro každé dva vrcholy $u, v \in V$ existuje v G orientovaná cesta z u do v a **současně** orientovaná cesta z v do u .



- Zjistit, zda je graf silně souvislý, bychom mohli tak, že spustíme **BFS** z každého vrcholu.
- Existuje ale výrazně efektivnější způsob, jak úlohu řešit (to se ale dozvíte až v BI-AG2).

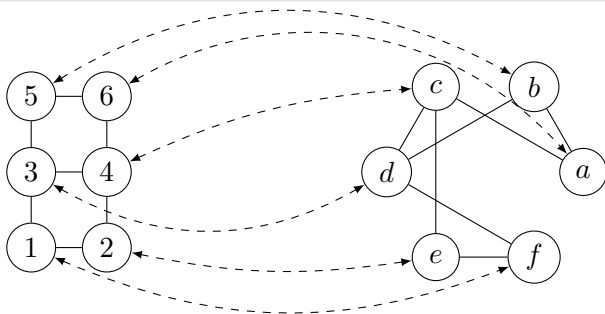
Izomorfismus grafů

Definice

Nechť G a H jsou dva grafy.

Funkce $f: V(G) \rightarrow V(H)$ je **izomorfismus grafů** G a H , pokud

- f je bijekce
- a pro každou dvojici vrcholů u, v z $V(G)$ platí $\{u, v\} \in E(G)$ právě tehdy, když $\{f(u), f(v)\} \in E(H)$.



Izomorfismus grafů

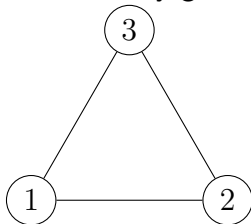
Definice

Dva grafy G a H jsou **izomorfní**, pokud existuje izomorfismus grafů G a H .

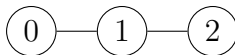
Tento fakt značíme $G \simeq H$.

Problém rozhodnout, zda dva grafy stejné velikosti jsou izomorfní, je výpočetně obtížný.

Avšak pro některé třídy grafů je efektivně řešitelný.



(a) $C_3 \simeq K_3$



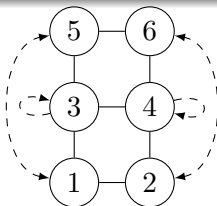
(b) $P_3 \simeq K_{1,2}$

Automorfismus grafu

Definice

Automorfismus grafu G je izomorfismus se sebou samým, tedy funkce $f: V(G) \rightarrow V(G)$ taková, že

- f je bijekce
- a pro každou dvojici vrcholů $u, v \in V(G)$ platí $\{u, v\} \in E(G)$ právě tehdy, když $\{f(u), f(v)\} \in E(G)$.



v	1	2	3	4	5	6
$f(v)$	5	6	3	4	1	2

Neformálně řečeno:

Automorfismus je permutace vrcholů, zachovávající „býti hranou“.

Automorfismy ukazují „symetrie“ grafu.

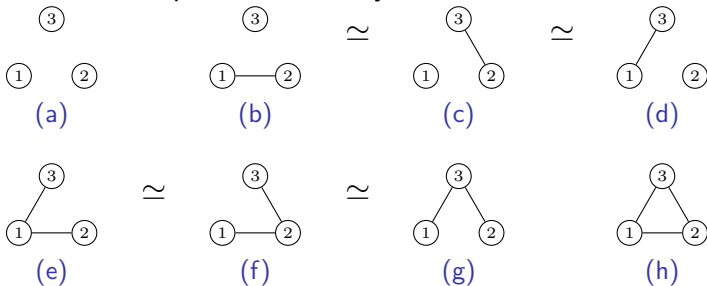
Množství automorfismů tedy ukazuje na míru pravidelnosti grafu.

Počty grafů

Fakt

Na n -prvkové množině vrcholů V je právě $2^{\binom{n}{2}}$ různých grafů.

- Navzájem neizomorfních grafů na V je méně.
- Např. pro $V = \{1, 2, 3\}$ dostáváme $2^{\binom{3}{2}} = 2^3 = 8$ všech grafů, z nichž ale jen 4 jsou neizomorfní.
- Pozn.: Binomiální číslo $\binom{n}{k}$ označuje počet k -prvkových podmnožin z n -prvkové množiny.



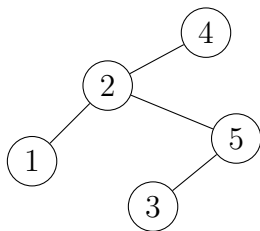
Strom, les, list

Definice

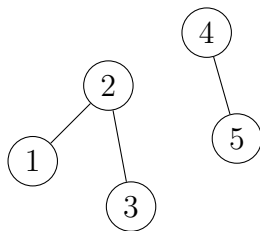
Graf G nazveme **stromem**, pokud je souvislý a neobsahuje žádnou kružnici (čili je acyklický).

Graf G nazveme **lesem**, pokud neobsahuje žádnou kružnici.

Vrchol v grafu G nazveme **listem**, pokud $\deg_G(v) = 1$.



(a) Strom se 3 listy



(b) Les

Základní vlastnost stromů je existence listů

Věta (o existenci listů)

Každý strom T s aspoň 2 vrcholy obsahuje aspoň dva listy.

Důkaz.

- Nechť P je nejdelší cesta v T .
- Nechť u a v jsou její koncové vrcholy.
- Sporem ukážeme, že u a v jsou listy.
- Kdyby z u vedly aspoň dvě hrany, jedna z nich bude patřit do u - v -cesty P a zbývat bude ještě aspoň jedna hrana $e = \{u, w\}$.
- Kdyby $w \in V(P)$, byla by v T kružnice.
- Kdyby $w \notin V(P)$, cesta P by šla prodloužit a nebyla by tedy nejdelší.
- To samé platí pro koncový vrchol v . □

Protože cesty jsou stromy, existují stromy, mající právě dva listy.

Listy, kružnice a souvislost v grafech I

- Uvažujme graf $G = (V, E)$, ve kterém existuje list v . Označme w jeho souseda v G . Pak **odebrání listu** v z G je operace, kterou vznikne graf $G - v = (V \setminus \{v\}, E \setminus \{\{w, v\}\})$.
- Uvažujme graf $G = (V, E)$ a zvolme $w \in V$. Pak **přidání listu** v **k vrcholu** w , $v \notin V$, ke grafu G je operace, kterou vznikne graf $G + v = (V \cup \{v\}, E \cup \{\{w, v\}\})$.

Věta (o trhání listů)

Nechť $G = (V, E)$ je graf na aspoň 2 vrcholech a nechť $v \in V$ je jeho list. Pak jsou následující tvrzení ekvivalentní:

- 1 G je strom.
- 2 $G - v$ je strom.

Pokud ze stromu list odebereme nebo do stromu list přidáme, zůstává stromem.

Listy, kružnice a souvislost v grafech II

① G je strom $\Rightarrow$ ② $G - v$ je strom.

Důkaz.

① $G - v$ je souvislý:

- * Zvol dva libovolně $x, y \in V \setminus \{v\}$ a cestu P mezi nimi v G (ta existuje, protože G je souvislý).
- * P nemůže obsahovat jiné vrcholy stupně 1 než x, y ; tedy speciálně neobsahuje v .
- * Dostáváme tedy $P \subseteq G - v$ a tedy $G - v$ je souvislý.

② $G - v$ je bez kružnic:

- * $G - v$ je podgraf G a tedy neobsahuje kružnici.

Tedy $G - v$ je strom.



Listy, kružnice a souvislost v grafech III

② $G - v$ je strom $\Rightarrow$ ① G je strom.

Důkaz.

① G je bez kružnic:

- * Vrácením listu v do grafu G (přidáním listu ke stromu $G - v$) zjevně nevytvoříme kružnici.

② G je souvislý:

- * Mezi každými dvěma $x, y \in V \setminus \{v\}$ vede cesta v $G - v$.
- * Cesta v G z libovolného vrcholu x do v vznikne prodloužením cesty mezi x a w do vrcholu v , kde w je (jediný) soused listu v .

Tedy G je strom. □



Charakterizace stromů

- Kombinace acykličnosti a souvislosti dává stromům několik unikátních vlastností a to umožňuje několik alternativních charakterizací stromů.

Věta (charakterizace stromů)

Nechť $G = (V, E)$ je graf. Pak následující tvrzení jsou ekvivalentní:

- 1 G je strom.
- 2 Pro každé dva vrcholy $u, v \in V$ existuje právě jedna u - v -cesta.
- 3 G je souvislý a vynecháním libovolné hrany vznikne nesouvislý graf.
- 4 G je souvislý a $|E| = |V| - 1$.

Co bychom po dnešku měli znát

- Použití BFS.
- Souvislý graf, souvislé komponenty.
- Výpočetní model RAM.
- Reprezentace (ne)orientovaných grafů.
- Nalezení souvislé komponenty, zjišťování souvislosti.
- Rozklad grafu na souvislé komponenty.
- Složitost BFS.
- Slabě a silně souvislý orientovaný graf.
- Izomorfismus a automorfismus grafů.
- Strom, les, list.
- Věta o existenci listů.